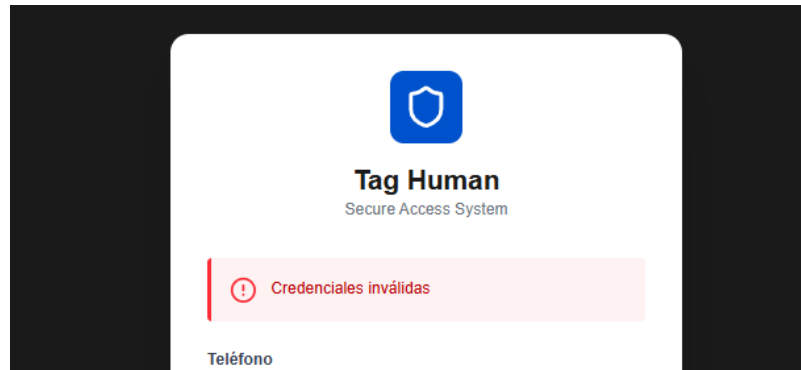


ANIMACIONES FUNCIONALES Y ACCESIBLES EN EL PROYECTO

- Señalar componentes donde agregaste animaciones con sentido:
- LoginScreen.jsx: aparición suave de errores.

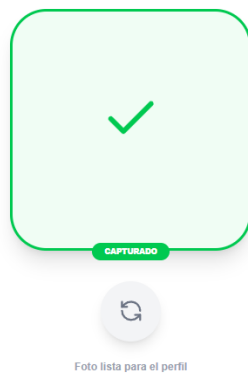


- RegisterStep1.jsx: barra de progreso con transición.

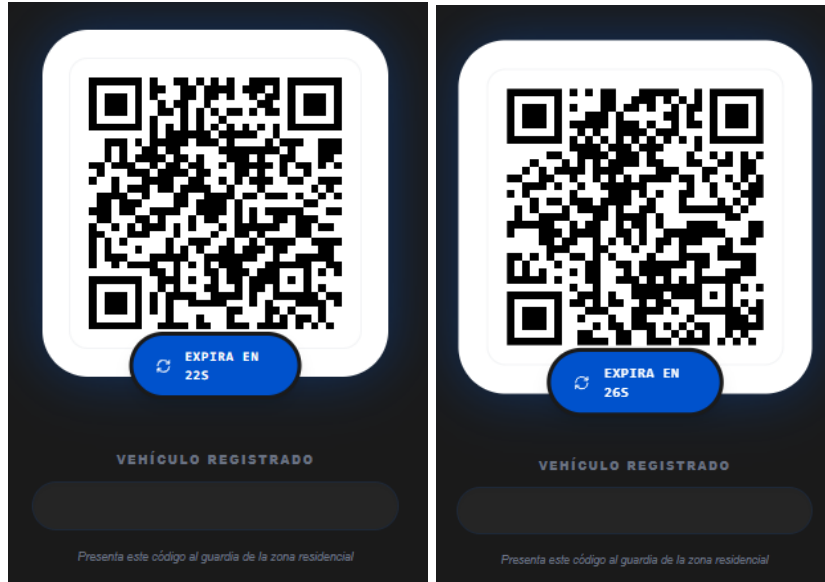
← Datos Personales

INFORMACIÓN

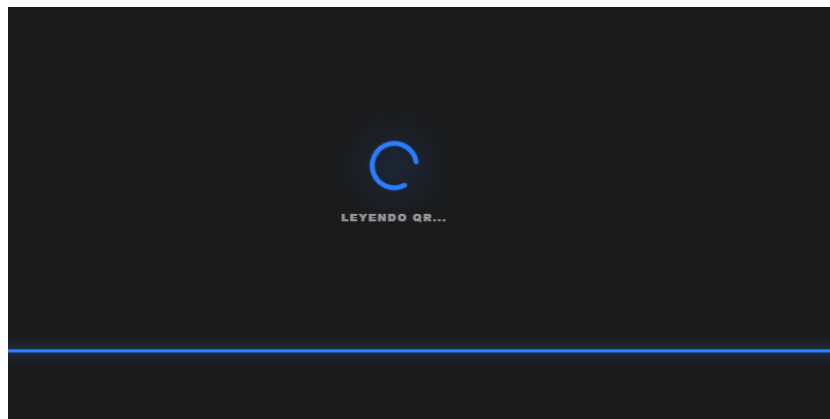
- RegisterStep2.jsx → estados de captura/loader/éxito.



- DriverDashboard.jsx → hover del contenedor de QR y animación del contador



- GuardScanner.jsx → loader, texto “Leyendo QR...” y línea de escaneo.



Evidencia de respeto a prefers-reduced-motion

- Ejemplos en el código:
 - Clases como motion-reduce:animate-none, motion-reduce:transition-none, motion-reduce:hover:scale-100 en:
 - RegisterStep1.jsx, RegisterStep2.jsx, DriverDashboard.jsx, GuardScanner.jsx, LoginScreen.jsx.

```

47   iv className="mb-6" aria-label="Progreso: paso 1 de 2" role="progressbar" aria-valuenow="50">
48     <div className="flex justify-between mb-2 text-xs font-bold text-gray-500 uppercase">
49       <span>Información</span>
50       <span>50%</span>
51     </div>
52     <div className="w-full h-2 bg-gray-100 rounded-full">
53       <div className="w-1/2 h-full bg-[#0052CC] rounded-full transition-all duration-500 motion-reduce:transition-none" />
54     </div>
55   </div>

```

```

    className={`w-full aspect-square max-w-xs rounded-[3rem] border-4 flex flex-col items-center justify-center`}
    ${captured ? 'border-green-500 bg-green-50' : 'border-dashed border-gray-200 bg-gray-50'}
  >
  {loading ? (
    <Loader2 className="w-12 h-12 text-[#0052CC] animate-spin motion-reduce:animate-none" />
  ) : captured ? (
    <Check className="w-20 h-20 text-green-500 animate-bounce motion-reduce:animate-none" />
  ) : (
    <Camera className="w-16 h-16 text-gray-300" />
  )}

```

```

<main className="flex-1 flex flex-col items-center justify-center">
  /* CONTENEDOR DE QR REAL */
  <div className="relative p-6 bg-white rounded-[2.5rem] shadow-[0_0_50px_rgba(0,82,204,0.4)] transition-transform hover:scale-105 motion-reduce:transition-none">
    <div className="p-4 bg-white rounded-2xl border-2 border-gray-100">
      /* Usamos QRCodeSVG para generar el QR con los datos del usuario */
      <QRCodeSVG

```

- Media query en GuardScanner.jsx:
- Regla @media (prefers-reduced-motion: reduce) { .animate-scan-line { animation: none; } }.

```

<style jsx>{`
  @keyframes scan-line {
    0% { top: 20%; }
    100% { top: 80%; }
  }
  .animate-scan-line {
    animation: scan-line 1.5s infinite alternate ease-in-out;
  }
  @media (prefers-reduced-motion: reduce) {
    .animate-scan-line {
      animation: none;
    }
  }
`}</style>

```

¿Por qué se usaron estas animaciones?

Las animaciones se diseñaron con tres propósitos principales:

1. Comunicar estado del sistema

El usuario necesita saber constantemente qué está ocurriendo: si el sistema está procesando, si su acción fue registrada o si debe esperar. Las animaciones ayudan a responder esas dudas sin necesidad de texto adicional.

2. Reforzar progreso y temporalidad

Elementos como barras de progreso, contadores o indicadores visuales permiten al usuario anticipar lo que ocurrirá, reduciendo incertidumbre.

3. Guiar la atención hacia lo importante

Movimientos sutiles (hover, aparición suave, rotaciones temporales) dirigen el foco hacia los elementos clave sin saturar la interfaz.

En ningún caso se implementaron animaciones meramente decorativas. Cada una cumple una función clara dentro del flujo.

Flujos donde se usan animaciones

1. Registro del repartidor (/register)

Durante el proceso de registro:

- La barra de progreso usa una transición suave para comunicar el avance del 50 % al 100 %.
- La captura de foto y los mensajes de error aparecen de forma progresiva, reforzando el feedback sin generar sobresalto visual.

Aquí la animación cumple un rol psicológico importante: reducir fricción en un flujo que puede generar tensión o incertidumbre.

2. Dashboard del conductor (/app/qr)

En esta pantalla crítica:

- El contenedor del QR tiene un leve efecto hover de escala, indicando que es el elemento principal.
- El icono de recarga gira únicamente cuando el QR está por expirar, reforzando urgencia temporal.

Estas animaciones ayudan a comunicar prioridad y tiempo restante sin necesidad de instrucciones adicionales.

3. Escáner del guardia (/guard/scanner)

En el escaneo:

- El loader y el texto “Leyendo QR...” se animan para confirmar que el sistema está procesando.
- La línea láser que recorre el visor refuerza visualmente el estado de “escaneando”.

Aquí la animación reduce la sensación de bloqueo y evita que el usuario repita acciones innecesariamente.

4. Login y validaciones

- Los mensajes de error aparecen con una animación suave de entrada.

Esto permite captar atención sin alterar el flujo ni cambiar el foco del teclado.

Accesibilidad: Respeto a prefers-reduced-motion

Un principio clave fue que no todos los usuarios toleran movimiento visual intenso.

Por ello se implementó soporte completo para prefers-reduced-motion mediante:

- motion-reduce:animate-none para desactivar animaciones como spin, bounce, pulse.
- motion-reduce:transition-none para eliminar transiciones en barras de progreso.
- motion-reduce:hover:scale-100 para evitar efectos de escala.
- En el escáner, una regla específica bajo @media (prefers-reduced-motion: reduce) que desactiva la animación de la línea láser.

De esta manera, el sistema se adapta automáticamente a usuarios con sensibilidad al movimiento o mareos, manteniendo funcionalidad sin comprometer accesibilidad.

Navegación por teclado y lectores de pantalla

Se aseguró que ninguna animación:

- Cambiara el orden de tabulación.
- Capturara o bloqueara el foco.
- Ocultara elementos esenciales para la navegación.

Además:

- Los botones y enlaces mantienen estados visibles de foco (focus:ring, anillos accesibles).
- Los errores usan role="alert".
- El temporizador del QR usa role="timer".
- Se emplea aria-live="polite" para anunciar cambios importantes sin interrumpir al usuario.

Esto garantiza compatibilidad con lectores de pantalla y navegación exclusivamente por teclado.

Criterios de diseño aplicados

1. Menos es más

Solo se animaron pantallas de alto impacto: registro, QR y escáner. No se animó cada elemento de la interfaz.

2. Estado por encima de espectáculo

Cada animación responde a una pregunta concreta del usuario:

- ¿Está cargando?
- ¿Mi acción se registró?
- ¿Cuánto tiempo queda?
- ¿Debo esperar?

3. Performance

Se priorizaron animaciones basadas en transform y opacity, optimizadas para GPU. Se evitaron reflows pesados y bucles innecesarios en componentes que se renderizan frecuentemente.

CI/CD pasando correctamente

```
PS C:\Users\josel\OneDrive\Documents\DWP\Frontend>tag-human-universal\frontend> npm run lint
> frontend@0.0.0 lint
> eslint .
```